

Build Automation & Dependency Management

Maven and Gradle fundamentals, and how dependencies & artifacts are managed in a modern build

IN THIS DOCUMENT

- Why build automation exists
- Maven overview & lifecycle
- Gradle overview & task model
- Maven vs Gradle comparison
- Dependency & artifact management concepts
- Transitive dependencies and version conflicts

Prepared for internal training · Topic 1 of 3

1 Build Automation Concepts

Why it matters: Manually compiling, testing, packaging, and deploying code is slow and error-prone. Build automation tools standardize this into repeatable, reliable steps that any developer or CI server can run the same way, every time.

Core Responsibilities of Any Build Tool

- **Compilation** — turning source code into bytecode/binaries
- **Dependency resolution** — pulling in the libraries the code needs
- **Testing** — running unit and integration tests as part of the build
- **Packaging** — producing a JAR, WAR, ZIP, or container image
- **Publishing** — pushing the finished artifact to a repository

Maven

Maven uses **XML** (`pom.xml`) and follows a convention-over-configuration philosophy. It has a fixed, well-known lifecycle:

```
validate → compile → test → package → verify → install → deploy
```

This rigidity is a feature: it makes builds predictable and standardized across large teams and organizations. Dependencies are declared in a `<dependencies>` block and resolved from a repository — first the local cache (`~/.m2`), then a remote repository such as Nexus or Maven Central.

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-core</artifactId>
  <version>5.3.20</version>
</dependency>
```

Gradle

Gradle uses a **Groovy or Kotlin DSL** (`build.gradle` / `build.gradle.kts`) and is programmable rather than purely declarative. It is **task-based** instead of following one fixed lifecycle, so custom build logic is easy to add. Gradle is generally faster than Maven because of **incremental builds** and a **build cache** — only what changed gets rebuilt. It's the dominant tool for Android and is increasingly common across modern JVM projects.

```
dependencies {  
    implementation 'org.springframework:spring-core:5.3.20'  
}
```

Maven vs Gradle — Quick Comparison

Aspect	Maven	Gradle
Config format	XML	Groovy / Kotlin DSL
Flexibility	Low (convention-driven)	High (scriptable)
Speed	Slower	Faster (caching, incremental builds)
Learning curve	Easier to start	Steeper, more powerful
Common use	Enterprise Java	Android, modern JVM, polyglot projects

2 Dependency & Artifact Management

Dependency — an external library your project needs (e.g. a JSON parser).

Artifact — the output your build produces (JAR, WAR, container image), or any versioned binary you consume or publish.

Coordinates

Every artifact is identified uniquely. Maven uses **GAV** coordinates: `groupId:artifactId:version`.

Example: `com.fasterxml.jackson.core:jackson-databind:2.15.0`.

Transitive Dependencies

A dependency's own dependencies get pulled in automatically. This is where version conflicts — often called "dependency hell" — happen. Resolution strategies differ by tool:

- **Maven** — "nearest wins" (the dependency closest to your project in the tree takes precedence)
- **Gradle** — explicit conflict resolution rules, and the ability to force a version

Where Dependencies Live: Repository Types

Repository	Purpose
Local repo	Cache on your own machine — <code>~/.m2</code> (Maven) or <code>~/.gradle/caches</code> (Gradle) Maven Central, npm registry, etc.

Remote / public
repo

Private / internal repo	Your organization's own artifacts, plus a cached copy of public ones — this is where Nexus comes in (see Part 2)
----------------------------	--

Good practice: pin dependency versions explicitly and use lock files (e.g. `gradle.lockfile`) or a dependency-management BOM to prevent version drift and "it works on my machine" problems.

Part 1 of 3 — Build Automation & Dependency Management. Continued in Part 2: Sonatype Nexus for Artifact Storage.